МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
**«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»**

# ЗВІТ
# З ЛАБОРАТОРНОЇ РОБОТИ № 8

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

**Тема:** **Тригери.**

**Мета:** навчитися створювати тригери.

## Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання – базу даних LibraryDB (варіант № 1 «Бібліотека»). Тригери створено командою CREATE TRIGGER; використано AFTER- і INSTEAD OF-тригери, віртуальні таблиці inserted і deleted, конструкції IF…ELSE та RAISERROR з відкатом транзакції. Результати виконання наведено на рисунках. Після зняття скріншотів змінені під час демонстрації дані відновлено до початкового стану.

### 1. Тригери для бази даних Sales111

У цій частині створено чотири тригери для бази даних Sales111: заборона оформлення замовлення після 25-го числа місяця, заборона видалення замовника із замовленнями, збільшення комісійних продавця за замовлення понад 1000 грн і заборона замовлень на суму менше 10 грн. Для кожного тригера наведено код створення та результат виконання, зокрема повідомлення про помилки під час відкату транзакції.

1.1. Створити тригер, який запускається під час вставки або модифікації замовлення і не дозволяє оформити його після 25-го числа місяця.

```
CREATE TRIGGER trg_Orders_Date ON Orders
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE DAY(DATE_OD) > 25)
    BEGIN
        RAISERROR(N'Замовлення можна оформлювати лише з 1 по 25 число місяця', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
GO

-- Дозволене замовлення: 15 число
INSERT INTO Orders (ID_OD, AMOUNT_OD, DATE_OD, ID_CM, ID_SP)
VALUES (3013, 500.00, '2024-03-15', 2001, 1003);

SELECT ID_OD, DATE_OD, AMOUNT_OD
FROM Orders
WHERE ID_OD = 3013;

-- Заборонене замовлення: 28 число
INSERT INTO Orders (ID_OD, AMOUNT_OD, DATE_OD, ID_CM, ID_SP)
VALUES (3014, 500.00, '2024-03-28', 2001, 1003);
```

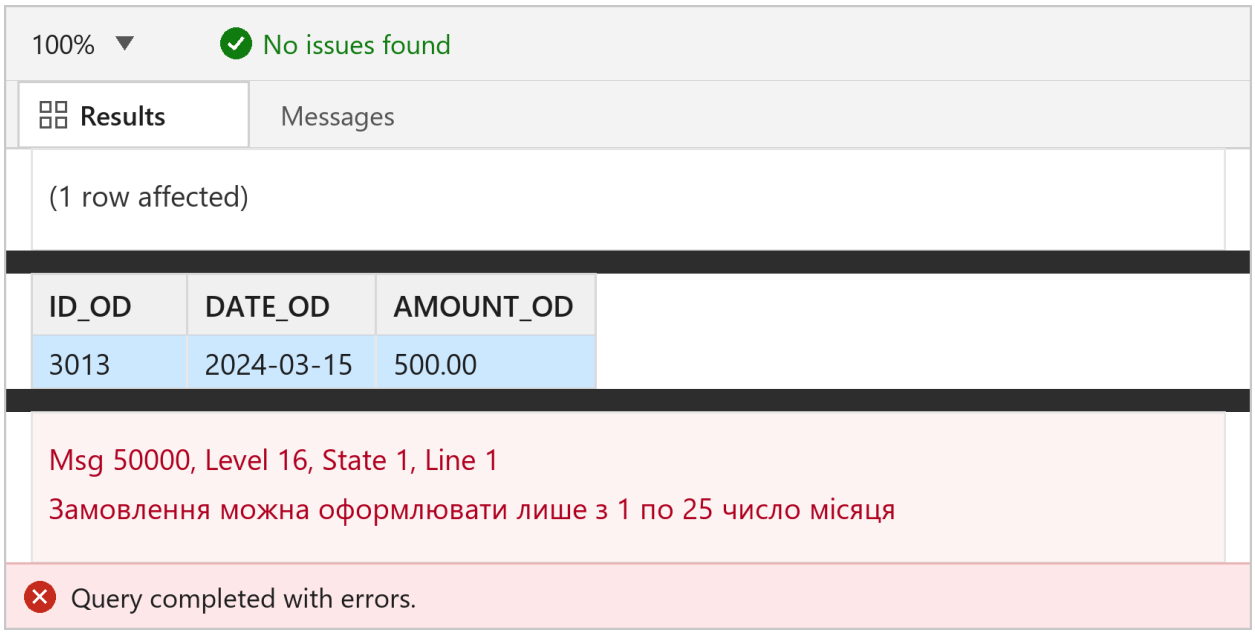



Рисунок 1 – Результат виконання тригера 1.1

*Замовлення 3013 на 15.03.2024 додано успішно. Замовлення 3014 на 28.03.2024 відхилено: тригер повернув помилку та відкотив транзакцію, тому рядок у таблиці Orders не з'явився. Після перевірки тестові рядки видалено.*

1.2. Створити тригер, який не дозволяє видалити замовника, якщо він має замовлення.

```
-- INSTEAD OF, оскільки зовнішній ключ fk_id_cm_Ord перевіряється раніше за AFTER-
тригер
CREATE TRIGGER trg_Customers_Delete ON Customers
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM deleted d
        JOIN Orders o ON o.ID_CM = d.ID_CM
    )
    BEGIN
        RAISERROR(N'Не можна видалити замовника, який має замовлення', 16, 1);
        RETURN;
    END;

    DELETE FROM Customers
    WHERE ID_CM IN (SELECT ID_CM FROM deleted);
END
GO

-- Клименко має замовлення – видалення заборонено
DELETE FROM Customers WHERE ID_CM = 2006;

-- Шевченко не має замовлень – видалення дозволено
DELETE FROM Customers WHERE ID_CM = 2005;

SELECT ID_CM, NAME_CM
FROM Customers
WHERE ID_CM IN (2005, 2006)
ORDER BY ID_CM;
```

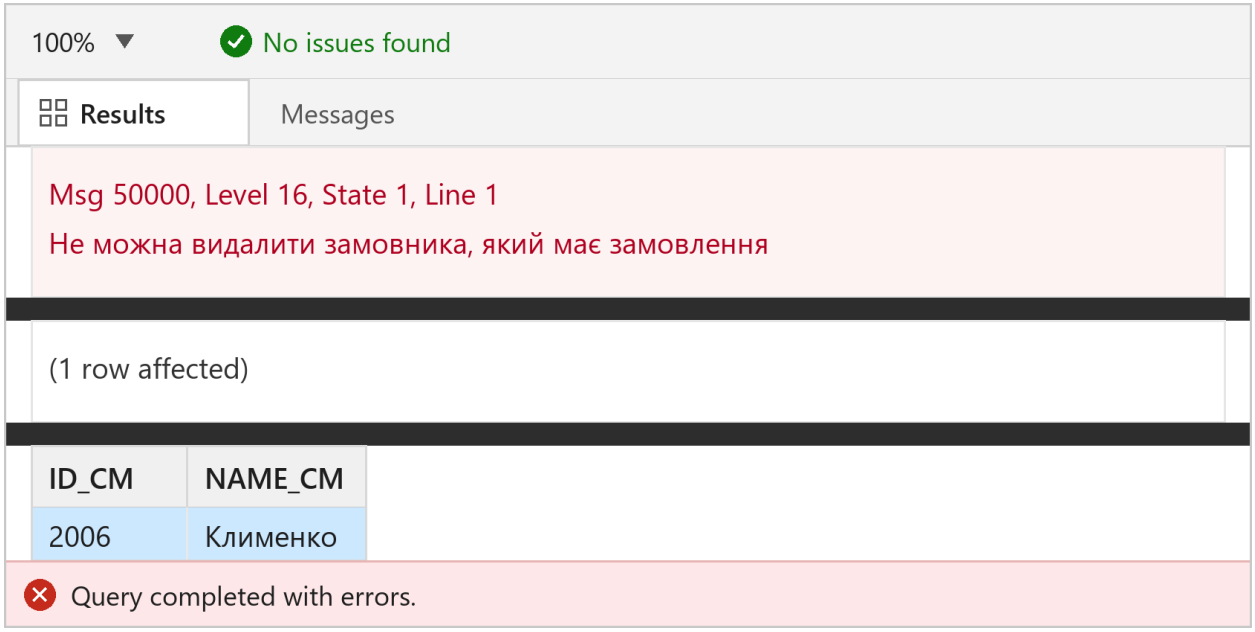



Рисунок 2 – Результат виконання тригера 1.2

*Замовника 2006 Клименка, який має замовлення, видалити не вдалося: тригер повернув помилку. Замовника 2005 Шевченка без замовлень видалено; після перевірки запис відновлено.*

*Використано INSTEAD OF DELETE, оскільки зовнішній ключ перевіряється до запуску AFTER-тригера.*

1.3. Створити тригер, що збільшує комісійні продавця на 0,01 під час додавання замовлення на суму понад 1000 грн.

```
CREATE TRIGGER trg_Orders_Commission ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE s
    SET COMM_SP = COMM_SP + 0.01 * x.cnt
    FROM Salespeople s
    JOIN (
        SELECT ID_SP, COUNT(*) AS cnt
        FROM inserted
        WHERE AMOUNT_OD > 1000
        GROUP BY ID_SP
    ) x ON x.ID_SP = s.ID_SP;
END
GO

-- Замовлення на 1500 грн
INSERT INTO Orders (ID_OD, AMOUNT_OD, DATE_OD, ID_CM, ID_SP)
VALUES (3013, 1500.00, '2024-03-15', 2001, 1003);

SELECT o.ID_OD, o.AMOUNT_OD, s.NAME_SP, s.COMM_SP
FROM Orders o
JOIN Salespeople s ON s.ID_SP = o.ID_SP
WHERE o.ID_OD = 3013;
```

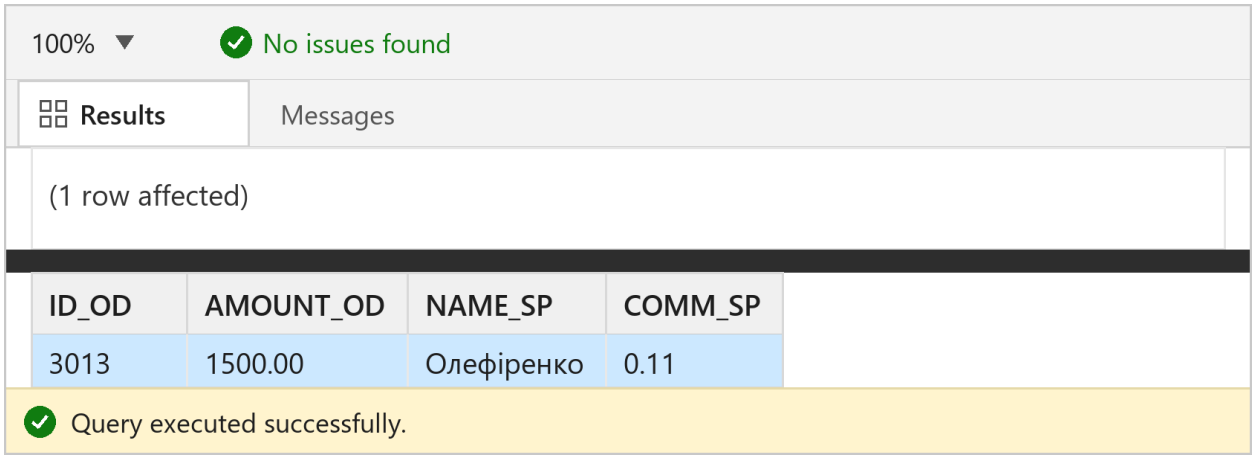



Рисунок 3 – Результат виконання тригера 1.3

*Після додавання замовлення 3013 на 1500 грн комісія продавця 1003 Олефіренка зросла з 0,10 до 0,11. Після перевірки замовлення видалено, комісію відновлено.*

1.4. Створити тригер, який забороняє додавати замовлення на суму, меншу за 10 грн.

```
CREATE TRIGGER trg_Orders_MinAmount ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE AMOUNT_OD < 10)
    BEGIN
        RAISERROR(N'Сума замовлення не може бути меншою за 10 грн', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
GO

-- Замовлення на 5 грн – заборонено
INSERT INTO Orders (ID_OD, AMOUNT_OD, DATE_OD, ID_CM, ID_SP)
VALUES (3014, 5.00, '2024-03-15', 2001, 1003);

SELECT COUNT(*) AS Кількість
FROM Orders
WHERE ID_OD = 3014;
```

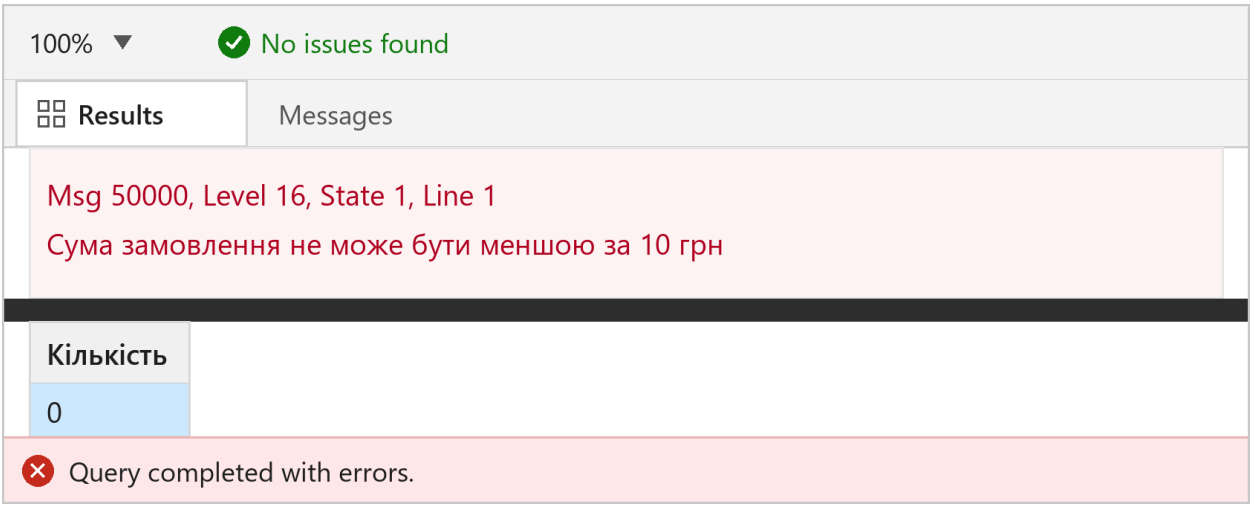



Рисунок 4 – Результат виконання тригера 1.4

*Замовлення 3014 на 5 грн відхилено: тригер повернув помилку та відкотив транзакцію. Запит до таблиці Orders підтверджує, що рядок не додано.*

## 2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 – предметна область «Бібліотека». Використано базу даних LibraryDB із таблицями Author, Publisher, Book, Reader і BookLoan. Для кожної таблиці створено три тригери – вставки, редагування та видалення (усього 15 тригерів). Усі тригери записують інформацію про операцію в службову таблицю AuditLog, що дозволяє переглянути історію змін. Для кожної таблиці виконано вставку, редагування та видалення тестового рядка, після чого показано вміст журналу.

## 2.1. Створити тригери вставки, редагування та видалення для таблиці Author.

```
CREATE TRIGGER trg_Author_Insert ON Author
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Author', N'INSERT', id_author, N'Додано автора: ' + author_name
    FROM inserted;
END
GO
CREATE TRIGGER trg_Author_Update ON Author
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Author', N'UPDATE', i.id_author, N'Країна: ' + ISNULL(d.country, N'—') +
N' -> ' + ISNULL(i.country, N'—')
    FROM inserted i JOIN deleted d ON d.id_author = i.id_author;
END
GO
CREATE TRIGGER trg_Author_Delete ON Author
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Author', N'DELETE', id_author, N'Видалено автора: ' + author_name
    FROM deleted;
END
GO
-- Демонстрація: INSERT, UPDATE, DELETE тестового рядка
INSERT INTO Author (author_name, country) VALUES (N'Тестовий автор', N'Україна');
UPDATE Author SET country = N'Польща' WHERE author_name = N'Тестовий автор';
DELETE FROM Author WHERE author_name = N'Тестовий автор';
SELECT table_name, action_name, row_id, details
FROM AuditLog WHERE table_name = N'Author' ORDER BY id_log;
```

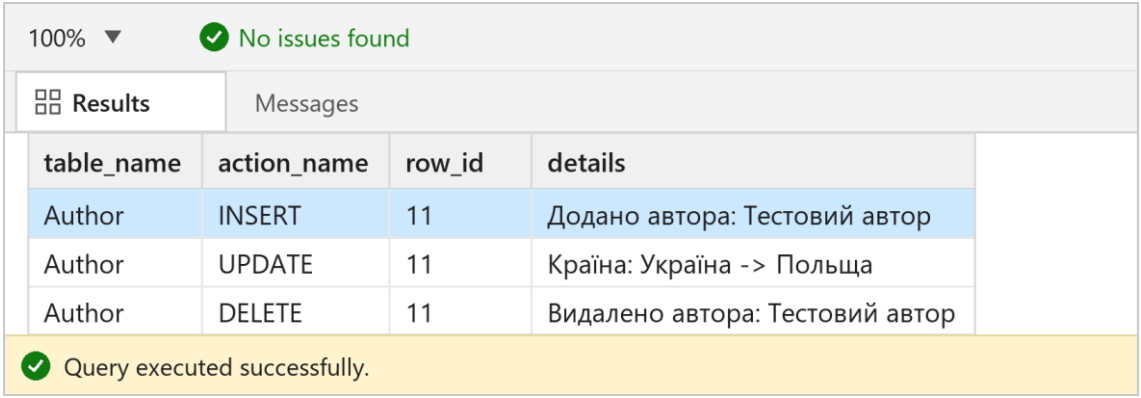



Рисунок 5 – Результат роботи тригерів таблиці Author

*Під час вставки, редагування та видалення тестового автора створено три записи в AuditLog. Тестовий рядок отримав код 11; після зняття скріншота його видалено, лічильник ідентичності повернуто до значення 10.*

## 2.2. Створити тригери вставки, редагування та видалення для таблиці Publisher.

```
CREATE TRIGGER trg_Publisher_Insert ON Publisher
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Publisher', N'INSERT', id_publisher, N'Додано видавництво: ' +
publisher_name
    FROM inserted;
END
GO
CREATE TRIGGER trg_Publisher_Update ON Publisher
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Publisher', N'UPDATE', i.id_publisher, N'Місто: ' + ISNULL(d.city, N'—') +
N' -> ' + ISNULL(i.city, N'—')
    FROM inserted i JOIN deleted d ON d.id_publisher = i.id_publisher;
END
GO
CREATE TRIGGER trg_Publisher_Delete ON Publisher
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Publisher', N'DELETE', id_publisher, N'Видалено видавництво: ' +
publisher_name
    FROM deleted;
END
GO
-- Демонстрація: INSERT, UPDATE, DELETE тестового рядка
INSERT INTO Publisher (publisher_name, city) VALUES (N'Тестове видавництво', N'Київ');
UPDATE Publisher SET city = N'Львів' WHERE publisher_name = N'Тестове видавництво';
DELETE FROM Publisher WHERE publisher_name = N'Тестове видавництво';
SELECT table_name, action_name, row_id, details
FROM AuditLog WHERE table_name = N'Publisher' ORDER BY id_log;
```

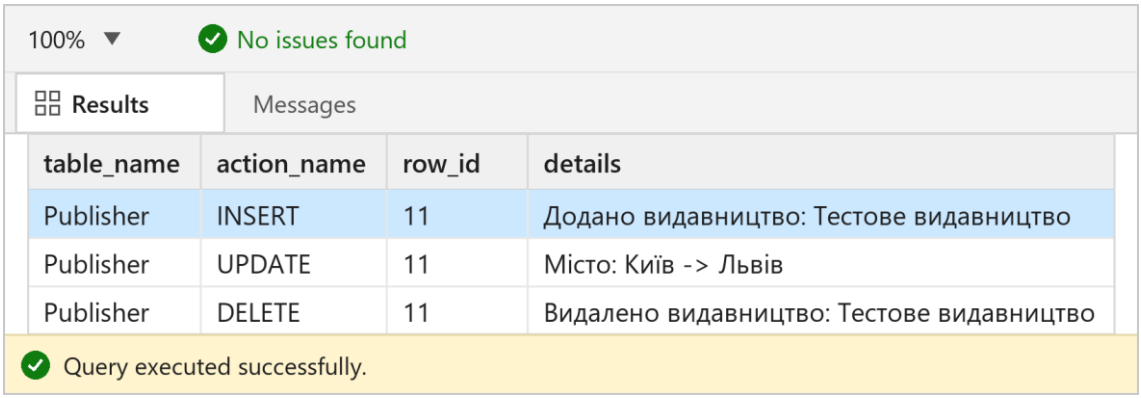



Рисунок 6 – Результат роботи тригерів таблиці Publisher

*Для видавництва зафіксовано вставку, зміну міста з Києва на Львів і видалення. Тестовий рядок отримав код 11; після перевірки його видалено, лічильник повернуто до 10.*

## 2.3. Створити тригери вставки, редагування та видалення для таблиці Book.

```
CREATE TRIGGER trg_Book_Insert ON Book
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Book', N'INSERT', id_book, N'Додано книгу: ' + title
    FROM inserted;
END
GO
CREATE TRIGGER trg_Book_Update ON Book
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Book', N'UPDATE', i.id_book, N'Ціна: ' + CAST(d.price AS NVARCHAR(20)) +
N' -> ' + CAST(i.price AS NVARCHAR(20))
    FROM inserted i JOIN deleted d ON d.id_book = i.id_book;
END
GO
CREATE TRIGGER trg_Book_Delete ON Book
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Book', N'DELETE', id_book, N'Видалено книгу: ' + title
    FROM deleted;
END
GO
-- Демонстрація: INSERT, UPDATE, DELETE тестового рядка
INSERT INTO Book (title, id_author, id_publisher, year_pub, price) VALUES (N'Тестова
книга', 1, 1, 2024, 100);
UPDATE Book SET price = 120 WHERE title = N'Тестова книга';
DELETE FROM Book WHERE title = N'Тестова книга';
SELECT table_name, action_name, row_id, details
FROM AuditLog WHERE table_name = N'Book' ORDER BY id_log;
```

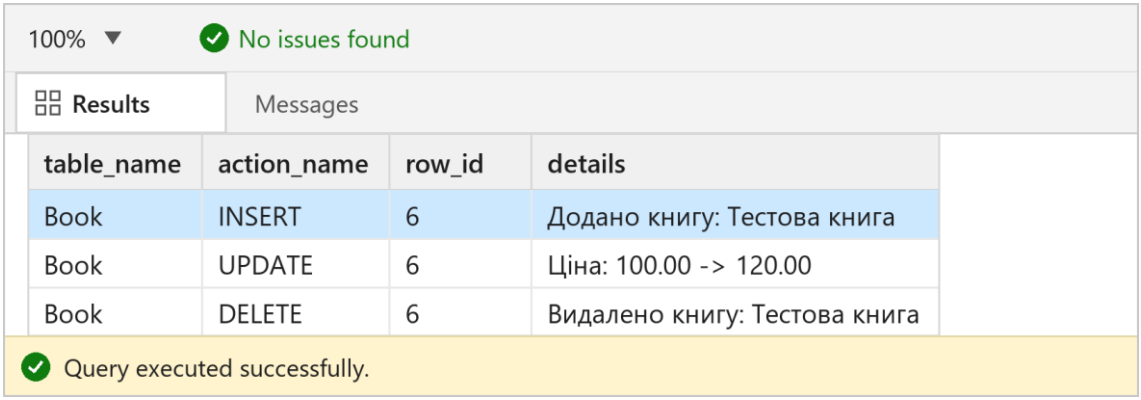



Рисунок 7 – Результат роботи тригерів таблиці Book

*Тестова книга отримала код 6; у журналі видно вставку, зміну ціни зі 100,00 на 120,00 та видалення. Після перевірки книгу видалено, лічильник повернуто до 5.*

## 2.4. Створити тригери вставки, редагування та видалення для таблиці Reader.

```
CREATE TRIGGER trg_Reader_Insert ON Reader
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Reader', N'INSERT', id_reader, N'Додано читача: ' + reader_name
    FROM inserted;
END
GO
CREATE TRIGGER trg_Reader_Update ON Reader
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Reader', N'UPDATE', i.id_reader, N'Телефон: ' + ISNULL(d.phone, N'—') + N'
-> ' + ISNULL(i.phone, N'—')
    FROM inserted i JOIN deleted d ON d.id_reader = i.id_reader;
END
GO
CREATE TRIGGER trg_Reader_Delete ON Reader
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Reader', N'DELETE', id_reader, N'Видалено читача: ' + reader_name
    FROM deleted;
END
GO
-- Демонстрація: INSERT, UPDATE, DELETE тестового рядка
INSERT INTO Reader (reader_name, ticket_num, phone) VALUES (N'Тестовий читач', N'T-
9999', N'+380000000000');
UPDATE Reader SET phone = N'+380501112233' WHERE ticket_num = N'T-9999';
DELETE FROM Reader WHERE ticket_num = N'T-9999';
SELECT table_name, action_name, row_id, details
FROM AuditLog WHERE table_name = N'Reader' ORDER BY id_log;
```

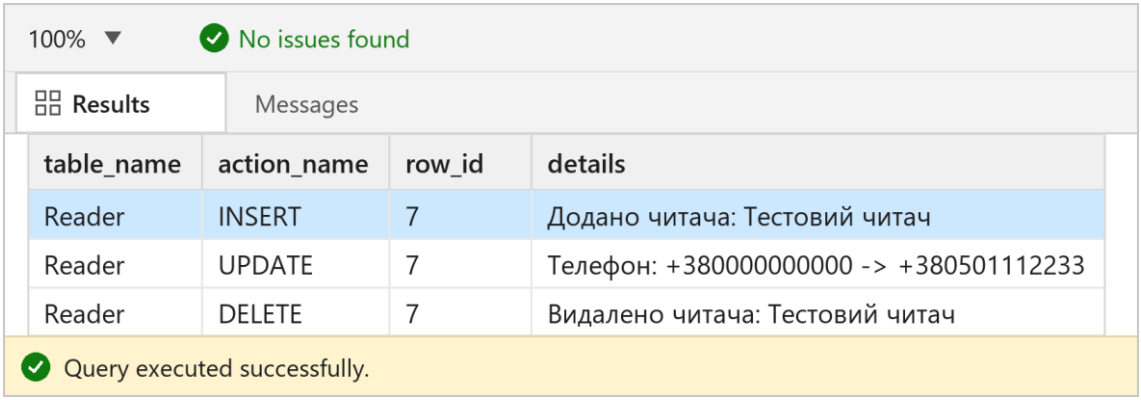



Рисунок 8 – Результат роботи тригерів таблиці Reader

*Тестовий читач отримав код 7; зафіксовано вставку, зміну телефону та видалення. Після перевірки рядок видалено, лічильник повернуто до 6.*

## 2.5. Створити тригери вставки, редагування та видалення для таблиці BookLoan.

```
CREATE TRIGGER trg_BookLoan_Insert ON BookLoan
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'BookLoan', N'INSERT', id_loan, N'Видано книгу ' + CAST(id_book AS
NVARCHAR(10)) + N' читачу ' + CAST(id_reader AS NVARCHAR(10))
    FROM inserted;
END
GO
CREATE TRIGGER trg_BookLoan_Update ON BookLoan
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'BookLoan', N'UPDATE', i.id_loan, N'Повернення: ' +
ISNULL(CONVERT(NVARCHAR(10), d.return_date, 120), N'—') + N' -> ' +
ISNULL(CONVERT(NVARCHAR(10), i.return_date, 120), N'—')
    FROM inserted i JOIN deleted d ON d.id_loan = i.id_loan;
END
GO
CREATE TRIGGER trg_BookLoan_Delete ON BookLoan
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'BookLoan', N'DELETE', id_loan, N'Видалено видачу книги ' + CAST(id_book AS
NVARCHAR(10)) + N' читачу ' + CAST(id_reader AS NVARCHAR(10))
    FROM deleted;
END
GO
-- Демонстрація: INSERT, UPDATE, DELETE тестового рядка
INSERT INTO BookLoan (id_book, id_reader, loan_date) VALUES (1, 1, '2026-10-01');
UPDATE BookLoan SET return_date = '2026-10-05' WHERE id_book = 1 AND id_reader = 1 AND
loan_date = '2026-10-01';
DELETE FROM BookLoan WHERE id_book = 1 AND id_reader = 1 AND loan_date = '2026-10-01';
SELECT table_name, action_name, row_id, details
FROM AuditLog WHERE table_name = N'BookLoan' ORDER BY id_log;
```

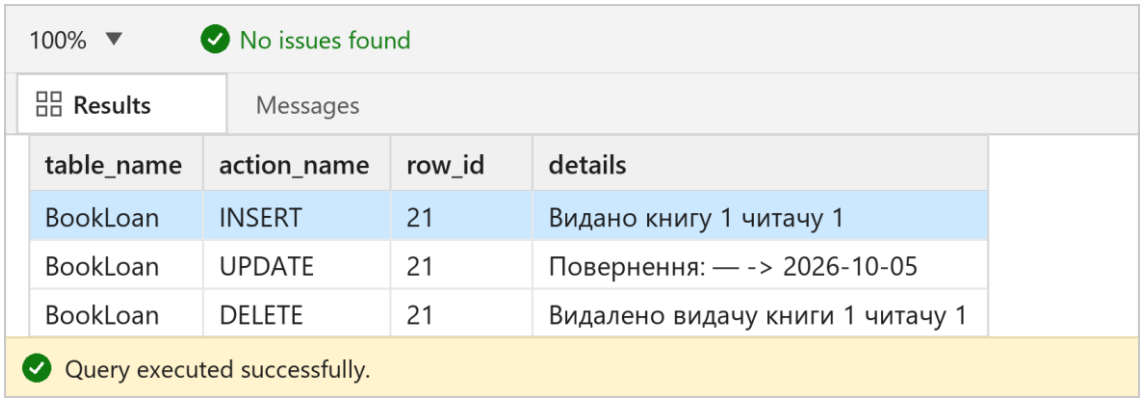



Рисунок 9 – Результат роботи тригерів таблиці BookLoan

*Тестова видача отримала код 21; у журналі зафіксовано вставку, встановлення дати повернення 2026-10-05 та видалення. Після перевірки видачу видалено, лічильник повернуто до 20.*

**Відповіді на контрольні питання**

1. Опишіть використання тригерів у MS SQL Server.

Тригер – це спеціальний об’єкт бази даних, який автоматично виконується у відповідь на подію зміни даних або іншу подію сервера. Тригери належать до збережених процедур особливого виду: вони не мають параметрів, їх не можна викликати командою EXEC, і користувач не може обійти їх виконання. Тригер спрацьовує незалежно від того, який застосунок або скрипт виконує зміну даних.

У MS SQL Server розрізняють такі види тригерів:

1) DML-тригери – реагують на INSERT, UPDATE, DELETE у таблиці або поданні. Вони бувають AFTER (FOR) – виконуються після успішної зміни та перевірки обмежень, і INSTEAD OF – замінюють операцію зміни даних;

2) DDL-тригери – реагують на події мови визначення даних (CREATE TABLE, ALTER TABLE, DROP TABLE тощо) в межах бази даних (ON DATABASE) або сервера (ON ALL SERVER);

3) тригери входу (LOGON) – виконуються під час встановлення сеансу користувача.

У DML-тригерах доступні дві спеціальні віртуальні таблиці: inserted – містить нові рядки (для INSERT і UPDATE), deleted – старі рядки (для UPDATE і DELETE). На відміну від звичайних таблиць, вони існують лише під час виконання тригера й мають структуру базової таблиці.

Тригери використовують для:

– перевірки складних бізнес-правил, які неможливо описати обмеженнями CHECK або зовнішніми ключами (наприклад, заборона оформлення замовлення після 25-го числа);

– каскадних змін у пов’язаних таблицях;

– ведення журналу змін (аудиту) – саме так зроблено в індивідуальному завданні цієї роботи;

– захисту від випадкового видалення або зміни важливих даних;

– формування власних повідомлень про помилки через RAISERROR і відкату транзакції.

Приклад тригера цієї роботи, який не дозволяє додати замовлення після 25-го числа:

```
CREATE TRIGGER trg_Orders_Date ON Orders
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE DAY(DATE_OD) > 25)
    BEGIN
        RAISERROR(N'Замовлення можна оформлювати лише з 1 по 25 число місяця', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
```

Переваги тригерів – автоматичне виконання, централізація складної логіки, можливість звертатися до інших таблиць і порівнювати стан до та після зміни. Недоліки – нижча швидкодія порівняно з обмеженнями, складність налагодження, неочевидність для розробника та ризик рекурсії. Тому основну цілісність рекомендується забезпечувати обмеженнями PRIMARY KEY, UNIQUE, CHECK і FOREIGN KEY, а тригери застосовувати лише для складних правил.

2. Як створити тригер? Обмеження на типи операторів, що використовуються при створенні тригерів. Наведіть приклад.

Тригер створюють командою CREATE TRIGGER (або CREATE OR ALTER TRIGGER у нових версіях). Загальний синтаксис:

```
CREATE [ OR ALTER ] TRIGGER schema_name.trigger_name
ON { table | view }
[ WITH <options> ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS
{ sql_statement [ ; ] [ ...n ] }
```

Команда CREATE TRIGGER має бути першим оператором пакета, тому в скрипті перед нею ставлять GO. Тригер прив’язують до конкретної таблиці або подання; для однієї події можна вказати кілька (наприклад, AFTER INSERT, UPDATE). Приклад із цієї роботи:

```
CREATE TRIGGER trg_Orders_MinAmount ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE AMOUNT_OD < 10)
    BEGIN
        RAISERROR(N'Сума замовлення не може бути меншою за 10 грн', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
GO
```

Обмеження та особливості:

1) тригер не має параметрів і не викликається безпосередньо командою EXEC;

2) тригер не повинен повертати результуючі набори (SELECT у тригері не рекомендується, оскільки клієнтська програма може не очікувати додаткового набору); для повідомлень використовують PRINT або RAISERROR;

3) не можна створювати DML-тригери для системних таблиць;

4) тригер не може змінювати структуру бази даних (для цього призначені DDL-тригери);

5) DML-тригер спрацьовує один раз на оператор, а не на кожен рядок, тому логіка має працювати з множинами рядків inserted і deleted;

6) INSTEAD OF-тригер для однієї події може бути лише один на таблицю або подання; AFTER-тригерів може бути кілька, порядок їх виконання задають процедурою sp_settriggerorder;

7) для створення потрібні права ALTER на таблицю та CREATE TRIGGER на базу даних;

8) тригер можна вимкнути командою ALTER TABLE ... DISABLE TRIGGER або видалити командою DROP TRIGGER.

DDL-тригери створюють на базі даних або сервері, наприклад:

```
CREATE TRIGGER trg_NoDropTable ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
BEGIN
    PRINT N'Зміна структури таблиць заборонена';
    ROLLBACK;
END
```

Такий тригер не дає випадково видалити або змінити таблицю бази даних.

3. Опишіть використання тригерів для операторів INSERT і UPDATE. Наведіть приклади.

Під час INSERT сервер поміщає нові рядки у віртуальну таблицю inserted, а під час UPDATE – старі версії рядків у deleted, нові – в inserted. Тригер може перевірити ці таблиці, порівняти значення до і після зміни, змінити дані в інших таблицях або відкотити транзакцію.

Найпоширеніші сценарії: перевірка допустимих значень, заборона зміни окремих стовпців (IF UPDATE), каскадне оновлення пов’язаних таблиць, ведення журналу змін, автоматичне коригування бонусів або комісійних.

Приклад 1 – заборона вставки або зміни замовлення після 25-го числа. Тригер перевіряє всі рядки inserted, тому працює і для багаторядкових операторів:

```
CREATE TRIGGER trg_Orders_Date ON Orders
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM inserted WHERE DAY(DATE_OD) > 25)
    BEGIN
        RAISERROR(N'Замовлення можна оформлювати лише з 1 по 25 число місяця', 16, 1);
        ROLLBACK TRANSACTION;
    END
END
```

Приклад 2 – збільшення комісійних продавця на 0,01 за кожне замовлення понад 1000 грн. Тригер групує рядки inserted за продавцем, тому для кількох замовлень комісія збільшується на відповідну кількість сотих:

```
CREATE TRIGGER trg_Orders_Commission ON Orders
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE s
    SET COMM_SP = COMM_SP + 0.01 * x.cnt
    FROM Salespeople s
    JOIN (
        SELECT ID_SP, COUNT(*) AS cnt
        FROM inserted
        WHERE AMOUNT_OD > 1000
        GROUP BY ID_SP
    ) x ON x.ID_SP = s.ID_SP;
END
```

Приклад 3 – журналювання змін. Для UPDATE використовують з'єднання inserted і deleted за первинним ключем, щоб показати старі та нові значення:

```
CREATE TRIGGER trg_Author_Update ON Author
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Author', N'UPDATE', i.id_author,
           N'Країна: ' + ISNULL(d.country, N'—') + N' -> ' + ISNULL(i.country, N'—')
    FROM inserted i JOIN deleted d ON d.id_author = i.id_author;
END
```

Якщо потрібно реагувати лише на зміну конкретного стовпця, застосовують функцію IF UPDATE(column_name). Вона повертає TRUE, якщо стовпець був указаний в операторі UPDATE, навіть якщо його значення не змінилося. Для перевірки саме зміни значення порівнюють inserted і deleted.

```
IF UPDATE(city_cm)
BEGIN
    PRINT N'Змінено місто замовника';
END
```

4. Опишіть використання тригерів для оператора DELETE. Наведіть приклад.

Під час DELETE видалені рядки поміщаються у віртуальну таблицю deleted. Тригер може перевірити, які саме рядки видаляються, і за потреби скасувати операцію командою ROLLBACK TRANSACTION або взагалі замінити її через INSTEAD OF DELETE.

Типові сценарії: заборона видалення пов'язаних записів, каскадне видалення дочірніх рядків, ведення журналу видалень, заборона масового видалення.

Приклад 1 – заборона видалення замовника, який має замовлення. Тут використано INSTEAD OF DELETE, оскільки зовнішній ключ fk_id_cm_Ord перевіряється до запуску AFTER-тригера й сам би заблокував видалення. Тригер перевіряє deleted і або повертає помилку, або виконує видалення:

```
CREATE TRIGGER trg_Customers_Delete ON Customers
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM deleted d
        JOIN Orders o ON o.ID_CM = d.ID_CM
    )
    BEGIN
        RAISERROR(N'Не можна видалити замовника, який має замовлення', 16, 1);
        RETURN;
    END;

    DELETE FROM Customers
    WHERE ID_CM IN (SELECT ID_CM FROM deleted);
END
```

Приклад 2 – журналювання видалення книги. Тригер зберігає код і назву видаленої книги в таблиці AuditLog:

```
CREATE TRIGGER trg_Book_Delete ON Book
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO AuditLog (table_name, action_name, row_id, details)
    SELECT N'Book', N'DELETE', id_book, N'Видалено книгу: ' + title
    FROM deleted;
END
```

Приклад 3 – каскадне видалення. Якщо видаляється автор, тригер може видалити його книги, а потім і видачі цих книг:

```
CREATE TRIGGER trg_Author_Delete_Cascade ON Author
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;
    DELETE FROM BookLoan
    WHERE id_book IN (SELECT id_book FROM Book
                      WHERE id_author IN (SELECT id_author FROM deleted));
    DELETE FROM Book
    WHERE id_author IN (SELECT id_author FROM deleted);
END
```

Особливості: тригер DELETE не спрацьовує на TRUNCATE TABLE, оскільки ця операція не записує окремі рядки в журнал транзакцій. Якщо в

таблиці є зовнішні ключі без каскадного видалення, AFTER-тригер може не встигнути виконатися – обмеження спрацює раніше. У такому разі застосовують INSTEAD OF DELETE або налаштовують каскадне видалення на рівні зовнішнього ключа.

**Висновки:**

Під час виконання лабораторної роботи № 8 було опановано створення та використання тригерів у MS SQL Server. Розглянуто команди CREATE TRIGGER, ALTER TRIGGER і DROP TRIGGER, види тригерів (AFTER, INSTEAD OF, DDL, LOGON), призначення віртуальних таблиць inserted і deleted, а також способи скасування недопустимих змін через RAISERROR і ROLLBACK TRANSACTION. Для бази даних Sales111 створено чотири тригери: trg_Orders_Date (заборона оформлення замовлення після 25-го числа), trg_Customers_Delete (заборона видалення замовника із замовленнями), trg_Orders_Commission (збільшення комісійних на 0,01 за замовлення понад 1000 грн) і trg_Orders_MinAmount (заборона замовлень на суму менше 10 грн). На їх прикладі показано успішні операції та відкат транзакцій із власними повідомленнями про помилки. Для бази даних LibraryDB (варіант № 1 «Бібліотека») створено по три тригери для кожної з п’яти таблиць – Author, Publisher, Book, Reader і BookLoan (усього 15 тригерів), які ведуть журнал операцій у таблиці AuditLog і фіксують значення до та після зміни. Усі тригери перевірено виконанням; тестові рядки видалено, лічильники ідентичності повернуто до початкових значень. Надано відповіді на всі чотири контрольні питання.